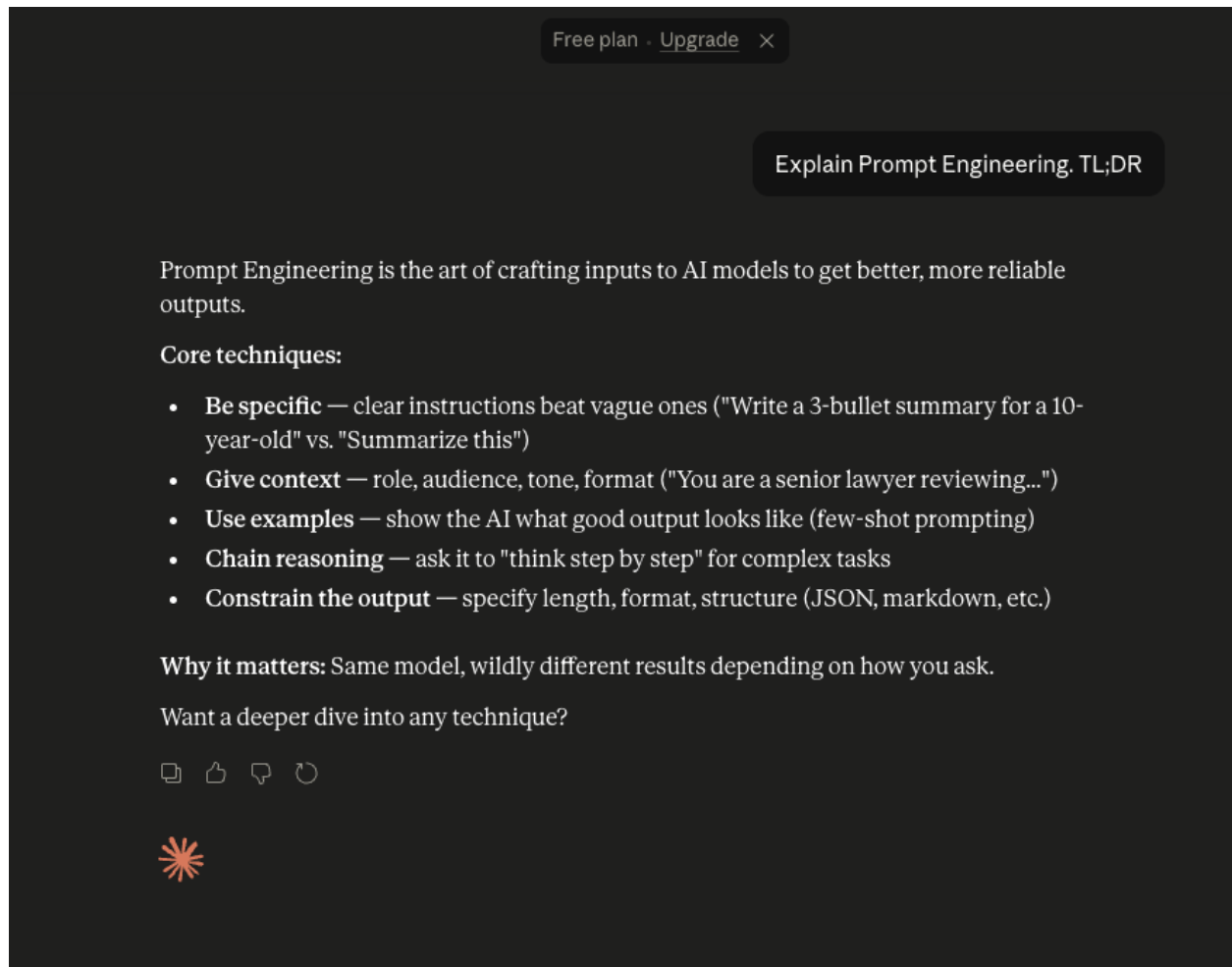


Part 1: TL;DR vs ELI-15 Prompting

Prompt A — TL;DR:



Prompt B — ELI-15:

Free plan · Upgrade

Explain Prompt Engineering. ELI-15

Prompt Engineering — Explained for a 15-year-old

Imagine you just got a **super-smart-but-literal** genie. It'll do *exactly* what you ask — but if you ask badly, you get a garbage answer.

Prompt Engineering = learning how to talk to the genie.

The basics

- **Vague ask** → "Write something about climate change" → you get a generic, boring essay
- **Smart ask** → "Write a punchy 5-tweet thread about climate change for Gen Z, using no jargon" → you get something actually useful

Key tricks

Trick	What it means	Example
Be specific	Tell it exactly what you want	"3 bullet points, under 20 words each"
Set the role	Give it a persona	"You are a sarcastic science teacher..."
Show examples	Demonstrate the style you want	"Write like this: [example]"
Think out loud	Ask it to reason step by step	"Explain your thinking before answering"
Constrain it	Limit format/length/tone	"Reply only in JSON"

The mental model

Think of AI like a **really powerful autocomplete** — it predicts what comes next based on what you gave it. Better input = better prediction = better output.

Prompt engineering is basically **hacking that prediction** in your favor.

The best part? You don't need to code. Just learn to communicate better with the AI — and you're ahead of most people.



Observation: TL;DR gives 2–3 bullet points/Concise answer

ELI-15 gives a friendly, analogy-based explanation with simpler words.

Part 2a: Automation Framework Architecture (Step-by-Step Prompting)

You are an Automation Architect.
[MANDATORY] Ask me 5 multiple choice questions one by one to understand my project requirements.
[STRICT] Wait for my answer before moving to the next question.
[MANDATORY] After all 5 answers, generate a complete Automation Framework Architecture.
[STRICT] Proceed step by step — show folder structure first, then explain each layer.
[STRICT] Upon my approval, proceed to the next implementation step.
Start with Question 1.

Great, waiting for your answer! 🌟



Question 1 of 5: What type of application are you automating? ✕

1 Web Application (Browser-based) ↗

2 Mobile App (iOS / Android)

3 API / Backend Services

4 Desktop Application

🔗 Something else

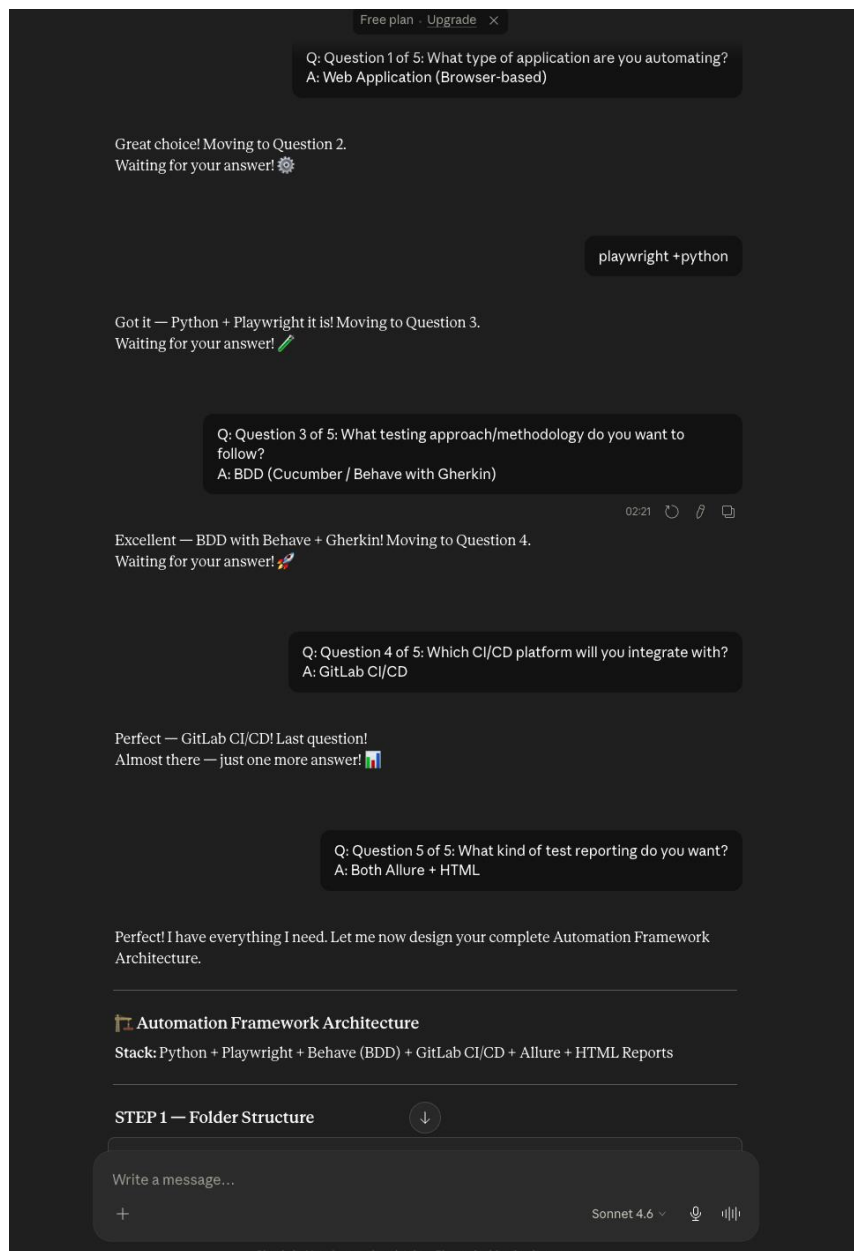
Skip

Or reply directly...



Sonnet 4.6 ▾





Perfect! I have everything I need. Let me now design your complete Automation Framework Architecture.

E Automation Framework Architecture

Stack: Python + Playwright + Behave (BDD) + GitLab CI/CD + Allure + HTML Reports

STEP 1 — Folder Structure

playwright_bdd_framework/

```
|
|
|— .gitlab-ci.yml      # GitLab CI/CD pipeline config
|— requirements.txt    # Python dependencies
|— pytest.ini         # Pytest / Behave config
|— behave.ini         # Behave runner config
|— allure-results/    # Raw Allure output (auto-generated)
|— reports/
|   |— allure-report/  # Final Allure HTML report
|   |— html-report/   # pytest-html report
|   |
|   |— features/      # BDD Layer (Gherkin)
|   |   |— login.feature
|   |   |— checkout.feature
|   |   |— environment.py    # Behave hooks (before/after scenario)
|   |
|   |— steps/         # Step Definitions Layer
|   |   |— login_steps.py
|   |   |— checkout_steps.py
|   |   |— common_steps.py
|   |
|   |— pages/         # Page Object Model (POM) Layer
|   |   |— base_page.py
|   |   |— login_page.py
|   |   |— checkout_page.py
|   |
|   |— utils/         # Utilities Layer
|   |   |— browser_manager.py    # Playwright browser setup/teardown
```

```

|   |— config_reader.py      # Reads config/env files
|   |— logger.py           # Centralized logging
|   |— screenshot_helper.py # Capture screenshots on failure
|
|   |— config/              # Configuration Layer
|   |   |— config.yaml      # Base config (URLs, timeouts)
|   |   |— dev.yaml         # Dev environment overrides
|   |   |— staging.yaml     # Staging environment overrides
|   |
|   |— test_data/          # Test Data Layer
|   |   |— login_data.json
|   |   |— checkout_data.json
|   |
|   |— drivers/            # Browser driver management (if needed)

```

STEP 2 — Layer-by-Layer Explanation

Layer	Folder/File	Responsibility
BDD Layer	features/	Gherkin .feature files written in plain English; environment.py manages hooks
Step Definitions	steps/	Maps Gherkin steps to Python functions; orchestrates page calls
Page Object Model	pages/	Encapsulates UI elements and actions per page; keeps tests clean
Utilities	utils/	Browser lifecycle, config reading, logging, screenshots
Configuration	config/	YAML-based env configs for dev/staging/prod
Test Data	test_data/	JSON files for data-driven scenarios within BDD
Reporting	reports/	Allure results + HTML output consumed by CI/CD
CI/CD	.gitlab-ci.yml	Pipeline: install → run tests → publish reports

Key Design Decisions

- **environment.py** acts as the test lifecycle controller — launches Playwright browser before each scenario, tears it down after

- **base_page.py** holds all shared Playwright actions (click, fill, wait) inherited by all page classes
- **browser_manager.py** centralizes headless/headed mode toggling, browser type selection (chromium/firefox/webkit)
- **Config YAML** allows environment switching via a single env variable — perfect for GitLab pipelines
- **Allure + HTML** both generated in the same pipeline run and published as GitLab artifacts

✅ **Folder structure and architecture are ready.**

Shall I proceed to **Step 3 — Implementation**? I'll generate the actual code for each layer, starting with:

1. requirements.txt + behave.ini + config.yaml
2. browser_manager.py + base_page.py
3. A sample feature file + step definitions + page object
4. .gitlab-ci.yml pipeline

Reply "Approved" to proceed! 🚀

Part 2b: Test Strategy Document (Chain of Thought Prompting)

Prompt:

Act as Senior Test Manager with 12+ years of experience Think step by step and reason through the best test strategy approach for my project before generating the final document. [CRITICAL] Then generate the complete Test Strategy Document.

PROJECT CONTEXT

Application Type : Web Based Application Automation Tool : Python + Playwright API
 Testing Approach : AI-Powered using OpenAI API (OpenAPI/Swagger docs fed to OpenAI LLM for intelligent API test generation) Team Size : 1 Resource AI Integration : OpenAI API Key integrated with LLM for auto-generating API test cases [MANDATORY] The Test Strategy Document MUST include:

1. Project Overview & Objectives

2. Scope (In-Scope & Out-of-Scope)
3. Test Approach
4. Test Levels (Unit, Integration, System, E2E, performance)
5. Test Types (Functional, Regression, Smoke, API, Exploratory)
6. AI-Powered API Testing Strategy - How OpenAPI/Swagger docs are fed to OpenAI LLM - How test cases are auto-generated using OpenAI API Key - Validation approach for AI generated test cases
7. Playwright + Python Framework Architecture - Folder structure - Page Object Model - Fixtures and Conftest - Reporting (Allure/HTML)
8. Resource Plan (Single Resource — task breakdown & priorities)
9. Entry & Exit Criteria
10. Risk & Mitigation Plan - Single resource risk - AI hallucination risk in test generation - OpenAI API rate limit risk
11. Test Deliverables
12. Tools & Tech Stack Summary [MANDATORY] Show Chain of Thought reasoning BEFORE the document. [STRICT] Output the document in a clean professional format. [TONE] Professional and precise.

AI-powered test strategy for web application

Free plan Upgrade

Now generating the full professional document as a .docx.

Run 5 commands, created a file, edited a file

Chain of Thought Reasoning (Pre-Documents)

Here is the step-by-step reasoning that shaped every decision in the document.

Step 1 — Constraint mapping. One resource means every decision must compound in value. Framework setup pays dividends each sprint; manual effort must be minimized ruthlessly. This drove the automation-first mandate and the AI pipeline investment upfront.

Step 2 — Testing pyramid calibration. Unit testing belongs to developers, QA ownership starts at integration (API layer) and goes upward. With Playwright available, E2E and functional coverage become the highest-leverage QA investment. The pyramid is skewed API-heavy + E2E — deliberately.

Step 3 — AI pipeline design. The OpenAI integration needed to be concrete and production-ready, not conceptual. The design landed on parse Swagger → chunk per endpoint → structured system prompt with explicit output schema → `response_format: json_object` for determinism → temperature 0.2 → three-gate validation (schema → contract → human). This prevents hallucination slipping into the suite.

Step 4 — Risk asymmetry. The two highest risks — single resource bus factor and AI hallucination — are asymmetric: low probability but catastrophic impact. The mitigations were designed to be structural (CI runs without human trigger; no AI test executes without review gate) rather than procedural, making them actually enforceable.

Step 5 — Deliverables realism. A solo QA resource cannot produce a 40-page test plan and a full suite simultaneously. Deliverables were scoped to high-value, non-redundant outputs: framework, AI pipeline, smoke suite, E2E suite, Allure reports, and a closure report. That's a realistic and complete set.

The Test Strategy Document is ready above — a fully formatted, professional .docx with all 12 mandatory sections, working code samples, color-coded risk matrix, architecture diagrams, and the complete Playwright POM framework structure. Everything from the AI pipeline prompt engineering to the sprint allocation table is production ready.

Test strategy document

Document .docx

Download

Next time, try this in CoWork

Hand off complex tasks for Claude to handle independently. Come back to finished results.

Get deeping now

Write a message...

Sonnet 4.6

Test strategy document .docx

TEST STRATEGY DOCUMENT | Web Application | Python + Playwright + AI | **CONFIDENTIAL — INTERNAL USE ONLY**

TEST STRATEGY DOCUMENT

Web Application | Python + Playwright | AI-Powered API Testing

Version	1.0
Status	Final
Date	May 2026
Classification	Internal Use Only
Prepared By	Senior Test Manager
Review Cycle	Per Sprint / Release